Université Ibnou Zohr

Faculté des Sciences d'Agadir

Centre d'Excellence IT

Module : Machine Learning / **Filière :** ADIA

Enseignant : Pr. Anass Garbaz

Travaux Pratiques N°8 : Introduction aux Systèmes de Recommandation

Introduction : Pourquoi les systèmes de recommandation ?

Netflix, Spotify, Amazon… Ces plateformes savent exactement quoi vous proposer. Derrière cette magie se cachent les systèmes de recommandation, l'un des cas d'usage les plus impactants du Machine Learning.

Objectif : Prédire la note qu'un utilisateur donnerait à un item qu'il n'a pas encore vu, afin de lui suggérer les items les plus pertinents.

Il existe deux grandes familles :

Approche	Principe	Exemple
Content-Based Filtering	Recommande des items similaires à ceux déjà appréciés	Si vous aimez Inception, vous aimerez Interstellar
Collaborative Filtering ✓	Recommande ce qu'ont aimé les utilisateurs similaires	Les gens comme vous ont adoré ce film

➢ **Dans cet atelier, nous nous concentrons sur le Collaborative Filtering.**

I - **Partie I :** Théorie

1.1 La matrice de notation

Le point de départ est une matrice **Utilisateurs × Films**. Chaque cellule contient la note donnée par un utilisateur à un film **(de 1 à 5 étoiles)**. La plupart des cellules sont vides, c'est exactement ce que nous devons prédire.

Exemple : 4 utilisateurs, 5 films (0 = n'a pas aimé, 5 = adore, ? = pas encore noté)

	Love at last	Romance Forever	Cute Puppies	Nonstop Shooting	Swords vs Karate
Alice	5	5	0	0	0
Bob	5	?	4	0	0
Carol	?	4	0	0	0
Dave	0	0	0	5	4

Observation clé : **Alice** et **Bob** aiment tous les deux les films romantiques. **Carol** aussi. On peut donc supposer qu'**Alice** aimerait le film que **Carol** a noté **4**. C'est l'intuition du **Collaborative Filtering** : les utilisateurs qui ont les mêmes goûts dans le passé auront tendance à avoir les mêmes goûts dans le futur

1.2 Paramétrage du modèle

On représente chaque film et chaque utilisateur par un vecteur de taille n (appelé vecteur latent) :

Notation	Signification
x(i)	Vecteur de caractéristiques du film i (appris automatiquement)
θ(j)	Vecteur de préférences de l'utilisateur j (appris automatiquement)
r(i,j) = 1	1 si l'utilisateur j a noté le film i, 0 sinon
y(i,j)	Note donnée par j au film i (uniquement si r(i,j) = 1)

1.3 Prédire une note : la formule clé

La note prédite par l'utilisateur **j** pour le film **i** est le produit scalaire entre son vecteur de préférences et le vecteur de caractéristiques du film :

$$\hat{y}(i,j) = \theta(j)^T \cdot x(i)$$

Exemple numérique (n=2) :

- **θ(Alice) = [4.5, 0.0] (aime le romantique, pas l'action)**
- **x(Love at last) = [0.9, 0.1] (très romantique, peu d'action)**
- **Prédiction = 4.5 × 0.9 + 0.0 × 0.1 = 4.05 ≈ 4 étoiles**

1.4 La fonction de coût

Pour apprendre les meilleurs θ(j) et x(i), on minimise l'erreur quadratique sur toutes les notes connues :

$$J = \frac{1}{2} \sum_{(i,j):r(i,j)=1} \left(\theta(j)^T x(i) - y(i,j)\right)^2 + \frac{\lambda}{2} \sum_j \sum_k \theta(j)_k^2 + \frac{\lambda}{2} \sum_i \sum_k x(i)_k^2$$

- ➢ Le premier terme mesure l'erreur de prédiction sur les notes connues.
- ➢ Les deux derniers termes sont la régularisation (évite le surapprentissage).
- ➢ λ est le paramètre de régularisation à régler.

1.5 Résumé de l'algorithme

- ➢ Initialiser **x(i) et θ(j)** avec de petites valeurs aléatoires.
- ➢ Minimiser J par descente de gradient (ou optimiseur Adam).
- ➢ **Pour prédire :** calculer **$\theta(j)^T \cdot x(i)$** pour toute paire **(i,j)** inconnue.
- ➢ Recommander les films avec les scores prédits les plus élevés.

II - Partie II : Exercices

Exercice 1 : Compréhension et calculs manuels

Contexte : On surveille les préférences de 5 utilisateurs sur 6 films. Les notes vont de 1 à 5 (? = non noté).

	The Notebook	Titanic	Mad Max	John Wick	Avatar	Inception
Alice	5	4	?	1	2	?
Bob	4	5	1	?	1	3
Carol	?	3	2	1	?	4
Dave	1	?	5	5	4	2
Eve	2	1	4	5	?	?

Q1. Comptez le nombre de notes connues **(r(i,j) = 1)** et le nombre de notes manquantes dans cette matrice.

Q2. En observant uniquement les notes d'**Alice** et de **Bob**, que pouvez-vous déduire sur leurs préférences respectives ? Quelle note **Bob** donnerait-il probablement à "**Avatar**" selon vous ? Justifiez votre réponse sans calcul, juste par intuition.

Q3. On fixe **n = 2 features (romantique, action).** On vous donne :

- ➢ **x(The Notebook) = [0.9, 0.05]**
- ➢ **x(Mad Max) = [0.1, 0.85]**
- ➢ **θ(Alice) = [4.8, 0.2]**
- ➢ **θ(Dave) = [0.1, 4.9]**

Calculez manuellement les 4 notes prédites suivantes et remplissez le tableau :

	The Notebook	Mad Max
Alice	?	?
Dave	?	?

Les prédictions sont-elles cohérentes avec les notes réelles du tableau original ? Commentez.

Q4. En vous basant uniquement sur les profils calculés en **Q3,** quelle note prédisez-vous pour **Alice** sur **"John Wick" (x(John Wick) = [0.05, 0.9]) ?** Ce film vaut-il la peine d'être recommandé à **Alice ?**

Exercice 2 : Implémentation python du collaborative filtering

Contexte : Vous allez implémenter un système de recommandation complet en Python avec NumPy et TensorFlow, en reprenant les données de l'exercice 1.

Q1. Mise en place

Créez la matrice de notation Y et le masque binaire R :

```
import numpy as np

Y_raw = [
   [5, 4, None, 1,    2,    None],  # The Notebook
   [4, 5, 1,    None, 1,    3   ],  # Titanic
   [None, 3, 2, 1,    None, 4   ],  # Mad Max
   [1, None, 5, 5,    4,    2   ],  # John Wick
   [2, 1,    4, 5,    None, None],  # Avatar
   [None, 3, None, 2, 3,    5   ],  # Inception
]

Y = np.array([[v if v is not None else 0 for v in row] for row in Y_raw], dtype=float)
R = np.array([[1 if v is not None else 0 for v in row] for row in Y_raw], dtype=float)

print("Shape de Y :", Y.shape)   # (6 films, 5 utilisateurs)
print("Notes connues :", int(R.sum()))
```

Vérifiez que le nombre de notes connues correspond à votre réponse de l'Exercice 1 - Q1.

Q2. Normalisation (mean centering)

Avant d'entraîner, centrez les notes de chaque film par sa moyenne sur les notes connues uniquement. Expliquez pourquoi cette étape est importante pour les nouveaux utilisateurs.

```
# Moyenne de chaque film sur les notes connues
Ymean = (Y * R).sum(axis=1) / (R.sum(axis=1) + 1e-8)
Ynorm = (Y - Ymean.reshape(-1, 1)) * R

print("Moyenne par film :")
```

```
films = ['The Notebook','Titanic','Mad Max','John Wick','Avatar','Inception']

for i, f in enumerate(films):
    print(f"  {f:15s} : {Ymean[i]:.2f}")
```

Q3. Initialisation des paramètres

Initialisez les matrices W (préférences utilisateurs) et X (features films) :

```
import tensorflow as tf

n_films, n_users = Y.shape
n_features = 2   # dimension latente

tf.random.set_seed(42)

W = tf.Variable(tf.random.normal([n_users, n_features], stddev=0.1), name='W')
X = tf.Variable(tf.random.normal([n_films,  n_features], stddev=0.1), name='X')
b = tf.Variable(tf.zeros([1, n_users]), name='b')
```

Quelle est la taille de W ? Quelle est la taille de X ? Que représente chaque ligne de ces matrices ?

Q4. Entraînement

Implémentez la boucle d'entraînement avec la descente de gradient Adam :

```
optimizer  = tf.keras.optimizers.Adam(learning_rate=0.1)
lambda_reg = 1.0
R_tf = tf.constant(R, dtype=tf.float32)
Y_tf = tf.constant(Ynorm, dtype=tf.float32)

for it in range(500):
    with tf.GradientTape() as tape:
        Y_pred = tf.linalg.matmul(X, W, transpose_b=True) + b
        error  = (Y_pred - Y_tf) * R_tf
        cost   = 0.5 * tf.reduce_sum(error**2)
        cost  += (lambda_reg/2) * (tf.reduce_sum(W**2) + tf.reduce_sum(X**2))

    grads = tape.gradient(cost, [W, X, b])
    optimizer.apply_gradients(zip(grads, [W, X, b]))

    if it % 100 == 0:
        print(f"Itération {it:4d} | Coût : {cost.numpy():.4f}")
```

Observez l'évolution du coût. Converge-t-il bien ? Que se passerait-il si le learning rate était trop grand ?

Q5. Génération des recommandations

```
Y_pred_final = (tf.linalg.matmul(X, W, transpose_b=True) + b).numpy()
Y_pred_final += Ymean.reshape(-1, 1)  # dé-normalisation
users = ['Alice', 'Bob', 'Carol', 'Dave', 'Eve']
```

```
print("\n=== Top 3 recommandations par utilisateur ===")
for j, user in enumerate(users):
    non_notes = np.where(R[:, j] == 0)[0]
    if len(non_notes) == 0:
        continue
    scores = Y_pred_final[non_notes, j]
    top3   = non_notes[np.argsort(scores)[::-1][:3]]
    print(f"{user:6s} → {[films[i] for i in top3]}")
```

Comparez les recommandations d'Alice et de Dave. Sont-elles cohérentes avec leurs profils calculés manuellement en Exercice 1 - Q3 ?

Q6. Analyse des features latentes

```
print("\nFeatures latentes apprises par le modèle :")
for i, film in enumerate(films):
    xi = X.numpy()[i]
    print(f"  {film:15s} : [{xi[0]:+.2f}, {xi[1]:+.2f}]")
```

Affichez aussi les features des utilisateurs **(W)**. Les films romantiques **(The Notebook, Titanic)** ont-ils des features similaires ? Les films d'action **(Mad Max, John Wick)** forment-ils un autre groupe ? Que peut-on conclure sur ce que le modèle a appris de façon non supervisée ?